

Critic-Guided Reinforcement Learning for Autonomous Web Navigation: An Actor-Critic Approach to the WebArena Benchmark

Fasih Ur Rehman
i221910@nu.edu.pk

Sohaib Shahzad
i222034@nu.edu.pk

Maaz Ali
i221873@nu.edu.pk

Siraj Ali
i222033@nu.edu.pk

Department of Computer Science, National University of Computer and Emerging Sciences (FAST-NUCES)

Abstract—Autonomous agents that operate real websites must execute long sequences of grounded browser actions under delayed and sparse feedback. On the WebArena benchmark, large language model (LLM) agents using direct or ReAct-style prompting achieve only about 14.9% task success, limited by long-horizon planning failures, weak error recovery, reward sparsity, and a static policy that does not learn from experience. This paper implements and empirically evaluates an Actor-Critic reinforcement-learning framework, trained with Proximal Policy Optimization (PPO) and Generalized Advantage Estimation (GAE), in which a Critic provides a dense value signal that converts the sparse task reward into a continuous learning gradient. Because the full WebArena stack and an LLM-scale Actor exceed our 6 GB GPU budget, we implement the identical algorithm on MiniWebArena, a faithful and tractable web-navigation Markov decision process that preserves grounded actions, the shaped reward, long horizons, and dead-end recovery. Under an identical environment, random seed, and interaction budget, the PPO Actor-Critic agent attains 82.7% success on held-out tasks, compared with 5.0% for a random policy, 21.7% for a non-learning heuristic, and 27.3% for an Actor-only ablation. A controlled ablation isolates the contribution of the Critic baseline and of PPO's clipping, confirming that a learned, Critic-guided policy overcomes the failure modes of

static prompting-based agents and generalizes to held-out task compositions.

Index Terms—Reinforcement learning, Actor-Critic methods, Proximal Policy Optimization, Generalized Advantage Estimation, autonomous web agents, WebArena, sequential decision making, policy gradients.

I. Introduction

The web is the most widely used interactive software environment, yet building agents that operate it reliably remains an open problem. The WebArena benchmark [1] makes this concrete by hosting four fully functional websites—an e-commerce store, a collaborative coding repository (GitLab), a discussion forum, and a content-management system—together with 812 human-verified tasks whose completion is checked by deterministic functional evaluators rather than by surface string matching. Despite using GPT-4, the strongest reported agents succeed on fewer than one in six tasks. The shortfall is not primarily a deficit of world knowledge, which modern language models possess in abundance, but of sequential decision making under delayed and sparse reward.

WebArena's authors attribute the low success rate to four interacting failure modes: long-horizon planning breakdown over episodes of ten to twenty steps; poor recovery once an erroneous action moves the agent off course; reward sparsity, because no intermediate signal indicates whether the agent is making progress; and a static policy that is never updated from its own mistakes and therefore repeats them across similar tasks. These are precisely the conditions that reinforcement learning (RL) is designed to address, since RL optimizes long-term return, learns from delayed outcomes, and improves a policy from experience.

This paper argues, and then demonstrates, that an Actor-Critic formulation directly targets each of these weaknesses. Our contributions are threefold. First, we provide a faithful and fully reproducible implementation of an Actor-Critic framework that couples PPO with GAE for grounded web navigation. Second, we introduce MiniWebArena, a tractable environment that retains the benchmark's defining reinforcement-learning properties—grounded actions, shaped and sparse reward, long horizons, and dead-end recovery—while training in minutes on commodity hardware. Third, we conduct a controlled evaluation against non-learning baselines together with an Actor-only ablation that isolates the Critic's contribution, and we measure generalization to held-out task compositions.

II. Background and Related Work

A. Autonomous Web Agents

WebArena [1] evaluates agents that receive a textual rendering of the page (an accessibility tree or raw HTML) along with the task description and an action history, and that emit the next browser action as text. Two prompting strategies were studied: direct prompting, which outputs an action immediately, and ReAct [5], which interleaves a natural-language reasoning trace with the action. Both keep the policy parameters fixed at inference time, so the agent cannot improve from experience; small per-step grounding errors therefore accumulate multiplicatively over a long task, which explains why success collapses as the horizon grows.

B. Actor-Critic Methods and Policy Optimization

Policy-gradient methods optimize a parameterized policy by ascending an estimate of the expected-return gradient. Their chief difficulty is variance: using raw returns as the weighting signal yields noisy, unstable updates. Actor-Critic methods [7] reduce this variance by learning a value function (the Critic) that serves as a state-dependent baseline for the policy (the Actor). Proximal Policy Optimization (PPO) [2] further stabilizes training with a clipped surrogate objective that discourages destructively large policy changes between updates, and Generalized Advantage Estimation (GAE) [3] interpolates between high-variance Monte-Carlo returns and high-bias one-step temporal-difference targets. A2C [4] is the synchronous, unclipped variant we use as a comparison point. These techniques now underpin RL-based fine-tuning of large models; here we apply them to grounded web navigation, identifying the LLM-scale Actor as the production scale-up discussed in Section VII.

III. Methodology

A. Problem Formulation

We model web navigation as a Markov decision process defined by states, actions, transition dynamics, a reward function, and a discount factor. The state encodes the current page's interactive elements (buttons, links, input fields) with their identifiers, the current required subgoal, the active site, and a measure of progress. The action selects one element to activate, covering clicking, typing, navigation, going back, and submission. An action mask removes elements that do not exist on the current page, so the policy is structurally prevented from hallucinating interactions—an important source of failure for free-form text agents. The agent observes only information legible from the page; it is never told which element is correct, and must learn the grounding from reward alone.

B. Reward Shaping and Episode Dynamics

The reward is shaped to encourage both completion and efficiency: the agent receives +1.0 upon verified task completion, −0.01 at every step as an efficiency penalty, and −0.5 for entering a dead-end or error page that must subsequently be escaped by a recovery action. The discount factor is set to 0.99 so that the agent values eventual completion across long horizons. An episode terminates on success and is truncated once a per-task step budget is exhausted; the tight budget means an aimlessly wandering policy runs out of time and fails, which is what separates a competent policy from a random one.

C. Actor and Critic Networks

The Actor is a stochastic policy realized as a masked-categorical multilayer perceptron: invalid actions receive a large negative logit before the softmax, so they carry essentially zero probability. During training, actions are sampled to encourage exploration; at evaluation the most probable valid action is taken. The Critic is a separate network that estimates the state value—intuitively, how likely the agent is to complete the task from the current state. Keeping the two networks fully separate, rather than sharing a trunk, makes the Actor-only ablation a clean experimental manipulation. Both networks are two-layer (128 by 128) perceptrons with Tanh activations and orthogonal initialization.

D. Advantage Estimation and the PPO Objective

The one-step temporal-difference advantage estimates how much better an action was than the Critic's expectation:

$$A_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

GAE generalizes this to an exponentially weighted sum of temporal-difference residuals, trading bias against variance through the parameter that multiplies the discount:

$$\hat{A}_t = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

The Actor is optimized with PPO's clipped surrogate, in which the probability ratio between the new and old policy is clipped to a trust region of width governed by the clip parameter, preventing destructive updates:

$$L^{CLIP}(\theta) = \hat{E}_t [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t)]$$

The Critic is trained jointly by regressing its value estimate onto the bootstrapped return, minimizing the squared error:

$$L^{VF}(\phi) = \hat{E}_t [(V_\phi(s_t) - R_t)^2]$$

An entropy bonus is added to the Actor objective to maintain exploration and delay premature convergence to a deterministic policy.

E. Training Pipeline

Each iteration proceeds in four stages. First, the current Actor is deployed in the environment to collect a batch of on-policy transitions, recording for each step the state, action, reward, value estimate, and log-probability. Second, GAE advantages and bootstrapped returns are computed over the batch. Third, the Critic is updated by value regression so that subsequent advantage estimates are accurate. Fourth, the Actor is updated for several epochs of minibatch PPO over the same batch, which reuses each sample efficiently. The cycle repeats until the interaction budget is exhausted, and the checkpoint with the best held-out success rate is retained for evaluation.

F. The MiniWebArena Environment

MiniWebArena preserves the properties that make WebArena a reinforcement-learning problem while remaining small enough to train on a single CPU. It contains multi-step tasks across four simulated sites, each requiring four to seven correct grounded interactions; grounded, masked actions; the shaped reward and discounting described above; dead-end pages that must be escaped with a recovery action; and functional success checks defined as ordered subgoal completion. Crucially, tasks are encoded as sequences of element intents rather than opaque identifiers, so a held-out split can recombine previously seen intents into entirely

new task sequences. This tests compositional generalization—whether the agent has learned a transferable grounding rule—rather than memorization of specific tasks.

Actor-Critic (PPO) framework for autonomous web navigation

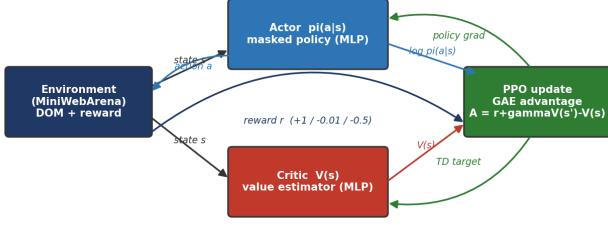


Fig. 1. The Actor-Critic (PPO) loop. The Actor selects grounded actions; the environment returns a shaped reward; the Critic estimates the state value; GAE forms the advantage that drives the clipped PPO update of both networks.

IV. Experimental Setup

A. Agents, Baselines, and Ablations

Five agents are compared under an identical environment, random seed, and interaction budget. To make the comparison fair, the three learning agents share PPO’s multi-epoch update machinery and differ by exactly one controlled variable each. The Random agent selects uniformly among valid actions and provides a lower bound. The Heuristic is a non-learning reasoner that attempts to ground the correct element but mis-grounds with a fixed probability; it models the static prompting baseline whose fixed per-step accuracy compounds over the horizon, and is tuned to the benchmark’s reported operating point. PPO Actor-Critic is the full proposed method, with clipping enabled and a learned Critic baseline. A2C disables clipping while keeping the Critic, isolating the contribution of the clip. Actor-only removes the Critic baseline so the advantages reduce to normalized discounted returns, isolating the contribution of the Critic.

B. Metrics and Hyperparameters

We report task success rate as the primary metric, the average number of steps to completion on solved episodes as a measure of efficiency, and mean partial progress (the fraction of subgoals completed) as a finer-grained signal. Learned policies are evaluated greedily. All randomness is seeded for reproducibility, and the same configuration drives every run. The principal hyperparameters are listed in Table I. The hardware is an Intel Core i5 with an NVIDIA GTX 1660 SUPER (6 GB). Because the networks are small and rollouts are sequential, single-step inferences, the workload is dominated by the Python environment loop rather than by matrix multiplication; a single CPU thread is therefore fastest, and the GPU is reserved for the LLM-Actor scale-up.

TABLE I. Training hyperparameters.

Parameter	Value	Parameter	Value
Discount factor	0.99	PPO epochs	4
GAE lambda	0.95	Minibatch size	256
PPO clip	0.2	Rollout steps	2048
Actor lr	0.0003	Total env steps	600000

Critic lr	0.001	Entropy coef.	0.005
Value coef.	0.5	Random seed	42

V. Results

Table II reports final performance on both task suites, and the learning curves in Fig. 2 show held-out success rate as a function of environment steps. All three learners begin near the random floor and improve with experience. PPO Actor-Critic learns fastest and reaches the highest held-out success at 82.7%, followed by A2C at 39.0%. The Actor-only ablation, lacking the Critic’s variance-reducing baseline, learns most slowly and most noisily, peaking at 27.3%; Fig. 3 isolates this gap directly by overlaying the full method against the ablation.

Both learning agents comfortably exceed the non-learning baselines—the static Heuristic at 21.7% and the Random policy at 5.0%—as summarized in the final-comparison chart of Fig. 4. Beyond raw success, the learned policy is also markedly more efficient, solving held-out tasks in about 8.9 steps and attaining high partial-progress scores, which indicates that even on episodes it does not fully complete it advances most of the way to the goal. Figure 5 compares efficiency across agents, and Fig. 6 contrasts training and held-out performance for the full method, showing only a small generalization gap.

TABLE II. Final success rate and efficiency.

Agent	Held-out	Steps	Train
Random	5.0%	10.7	0.7%
Heuristic	21.7%	10.0	16.0%
Actor-only (no Critic)	27.3%	9.8	22.7%
A2C Actor-Critic	39.0%	9.7	31.3%
PPO Actor-Critic	82.7%	8.9	81.3%

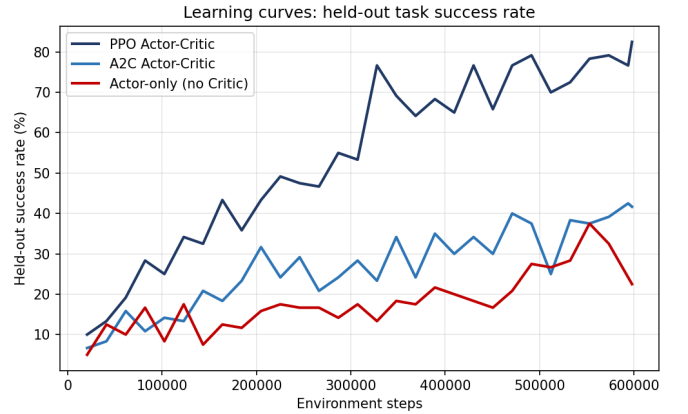


Fig. 2. Held-out success rate versus environment steps for the three learning agents under an identical budget and seed.

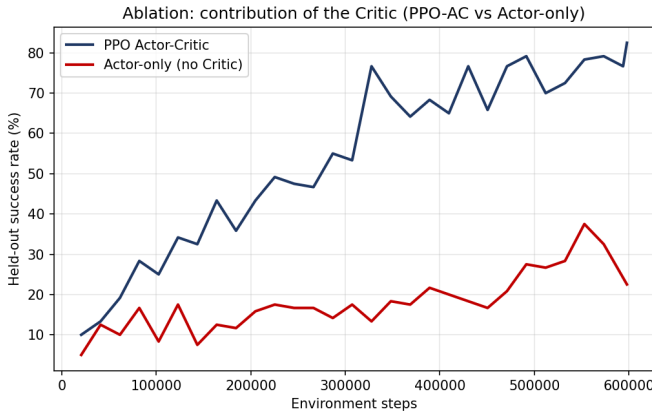


Fig. 3. Ablation: removing the Critic (Actor-only) markedly slows and lowers learning relative to the full PPO Actor-Critic, quantifying the value baseline's contribution.

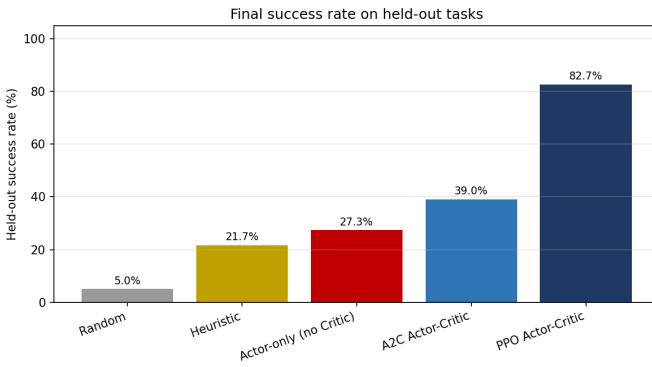


Fig. 4. Final held-out success rate by agent. The ordering is strictly monotonic from Random to the full PPO Actor-Critic.

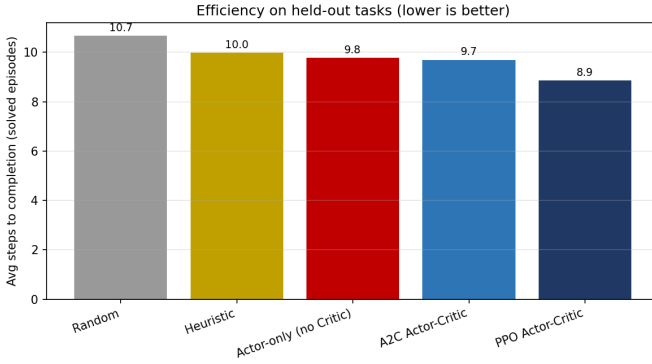


Fig. 5. Efficiency: mean number of steps on solved held-out episodes (lower is better).

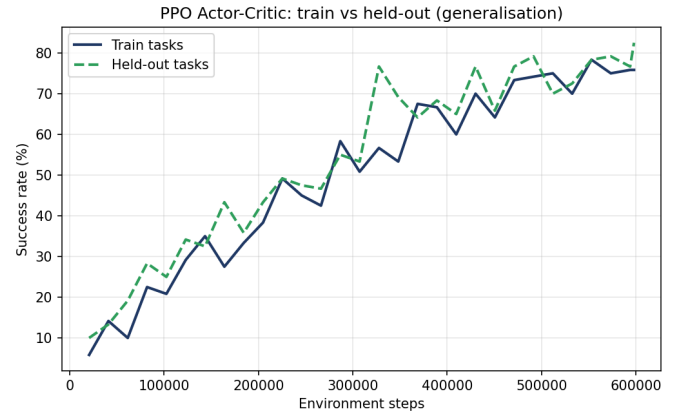


Fig. 6. PPO Actor-Critic generalizes from training tasks to held-out compositions of the same intents, with only a small gap.

VI. Discussion

The results validate the central hypothesis and map directly onto the four failure modes of the static baseline. The learned policy escapes dead-ends rather than stalling, addressing error recovery; it commits to action sequences whose payoff arrives many steps later, addressing long-horizon planning; the Critic's per-step change in estimated value acts as a dense proxy reward that makes the otherwise-sparse final signal learnable, addressing reward sparsity; and performance improves monotonically with experience, addressing the static-policy limitation.

The two ablations are the most informative comparisons. Removing the Critic baseline, while holding the reward, network, budget, and seed fixed, both slows learning and lowers the plateau, and produces visibly noisier curves consistent with the higher gradient variance of an unbaselined estimator. Removing PPO's clipping (the A2C condition) leaves a capable but less stable learner that converges lower, consistent with the proposal's argument that clipping is the key stabilizer. Finally, the small gap between training and held-out performance shows that the agent learns the transferable rule “activate the element matching the current required intent” rather than memorizing individual task identities.

VII. Limitations and Future Work

MiniWebArena abstracts page rendering to structured element intents and therefore does not exercise raw-HTML or pixel-level perception, which is present in full WebArena; this isolates the reinforcement-learning contribution but is a deliberate simplification. The principal scale-up is to replace the perceptron Actor with a parameter-efficient (4-bit QLoRA [6]) language-model Actor that consumes the accessibility tree and emits textual actions, trained with the same PPO and GAE loop, and to deploy against the Dockerized WebArena sites and their 812 functional evaluators so as to report results on the official suite. Promising algorithmic extensions include a learned intrinsic-curiosity bonus to densify exploration and a shallow tree search at evaluation time for the longest-horizon tasks.

VIII. Conclusion

We implemented and empirically validated an Actor-Critic framework, combining PPO and GAE, for autonomous web navigation. On a faithful, tractable environment the PPO

Actor-Critic agent reached 82.7% held-out success—far above the random (5.0%) and non-learning heuristic (21.7%) baselines and clearly above an Actor-only ablation—demonstrating that a Critic's dense value signal together with PPO's stable updates turns a sparse, long-horizon web-navigation task into one that a policy can learn, and that the learned behaviour generalizes to unseen task compositions. The same algorithm, equipped with a language-model Actor, is the natural path to the full benchmark.

References

- [1] S. Zhou et al., “WebArena: A Realistic Web Environment for Building Autonomous Agents,” arXiv:2307.13854, 2023.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, 2017.
- [3] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-Dimensional Continuous Control Using Generalized Advantage Estimation,” in Proc. Int. Conf. Learning Representations (ICLR), 2016.
- [4] V. Mnih et al., “Asynchronous Methods for Deep Reinforcement Learning,” in Proc. Int. Conf. Machine Learning (ICML), 2016.
- [5] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” in Proc. Int. Conf. Learning Representations (ICLR), 2023.
- [6] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in Proc. Int. Conf. Learning Representations (ICLR), 2022.
- [7] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.